

Module 1: Python Basics & Flow Control and Functions

1. Explain the basic structure of a Python program with an example.
2. What are the different data types supported by Python? Explain with examples.
3. How does Python handle string concatenation and replication? Illustrate with code.
4. Explain how variables are declared and used in Python.
5. Write a Python program to input two numbers and display their sum, difference, and product.
6. What is the role of the interactive shell in Python? Give examples.
7. Differentiate between integer and floating-point numbers in Python.
8. Write a Python program to demonstrate Boolean operators.
9. How does Python evaluate comparison operations? Write suitable examples.
10. Explain the flow control structure if-elif-else with a practical program.
11. Write a program using while loop to display numbers from 1 to 10.
12. What is the use of sys.exit() in Python? Illustrate with an example.
13. Describe the use of functions in Python. How do you define and call a function?
14. Write a function in Python to calculate the factorial of a number using recursion.
15. What is the significance of return statement in Python functions? Provide an example.
16. Explain the concept of global and local scope with code.
17. What is the purpose of the global statement in Python? When should it be used?
18. Demonstrate exception handling in Python using try-except block.
19. Explain the importance and usage of the None value in Python.
20. Write a short Python program to guess a number between 1 to 10.

Module 2: Lists, Tuples, and Dictionaries and Structuring data

1. What is a list in Python? How is it different from a string?
2. Write a Python program to add, remove, and update elements in a list.
3. Explain the use of augmented assignment operators with list examples.
4. How are list slicing and indexing used? Give examples.
5. Write a Python program to reverse a list without using built-in functions.
6. Describe any five list methods with syntax and examples.
7. What is the role of the append() and extend() methods? Illustrate.
8. Write a Python program that implements a Magic 8 Ball using a list.
9. Differentiate between a list and a tuple with code examples.
10. What are the advantages of using tuples in Python?
11. Explain how references work in Python with respect to lists.
12. Write a program to demonstrate aliasing and cloning of lists.
13. What are dictionaries in Python? How are they declared and used?
14. Explain the difference between keys(), values(), and items() methods.

15. Write a program to store and retrieve student details using dictionaries.
16. What is pretty printing in Python? How is it performed?
17. Demonstrate dictionary operations: insertion, deletion, and update with examples.
18. Compare and contrast lists and dictionaries in Python.
19. How can you model real-world entities like a contact book using data structures in Python?
20. Write a program using nested dictionaries to store department-wise student data.

Module 3: Manipulating Strings and Reading and Writing Files.

1. What is a string in Python? How is it different from a list or tuple?
2. Explain any five useful string methods with syntax and examples.
3. Write a Python program to count vowels, consonants, and digits in a string.
4. How do string slicing and indexing work? Give examples.
5. What is the purpose of the join(), split(), and replace() string methods?
6. Write a program to implement a password locker using Python.
7. Explain how to add bullets to Wiki markup using string operations in a Python project.
8. Describe the process of reading from and writing to a file in Python.
9. Write a program that reads a text file and displays its content line by line.
10. What are the different file modes in Python? Give suitable examples.
11. How does the os.path module help in file handling? Explain with examples.
12. Write a program that creates a file, writes data into it, and then reads it.
13. What is the difference between read(), readline(), and readlines() functions?
14. How does the with statement improve file handling in Python?
15. Explain how to save and retrieve variables using the shelve module.
16. What is the print.format() function used for? Provide examples of formatting strings.
17. Write a Python script to format output for a table of student marks.
18. Describe how to save variables using the shelve module and the print.format() method.
19. Write a Python program to generate random quiz files.
20. Describe the working of the multclipboard project and its real-world application.

Module 4: Organizing files and debugging

1. What is the purpose of the shutil module in Python? List any four commonly used functions.
2. Write a Python script to copy and rename files using the shutil module.
3. How does os.walk() help in walking through a directory tree? Give an example.
4. Write a program to display all files in a given directory and its subdirectories.
5. How do you compress files using the zipfile module in Python?
6. Explain the steps to create a ZIP file and add multiple files to it using Python.
7. Write a program to back up a folder into a ZIP file.
8. Describe a project that renames files from American-style dates to European-style dates.
9. How would you rename multiple files in a directory using a script?

10. What are exceptions in Python? How do you handle them using try-except blocks?
11. Write a Python program to handle multiple exceptions with appropriate messages.
12. What is the use of the raise statement in Python? Provide an example.
13. How can you get a traceback as a string? Explain with code.
14. What is the role of assert statements in debugging Python programs?
15. Differentiate between syntax errors, exceptions, and logical errors.
16. What is logging in Python? How is it different from using print statements?
17. Write a Python program that logs error messages to a file.
18. What are the different logging levels available in Python?
19. Explain how IDLE's debugger helps in debugging Python code.
20. List and explain three features of Python that help in identifying and fixing bugs quickly.

Module 5: Classes and objects, classes and functions, classes and methods

1. What are programmer-defined types in Python? Explain with an example.
2. Define a class and create objects using it.
3. What are attributes in Python classes? How are they accessed and modified?
4. Write a class Rectangle with methods to calculate area and perimeter.
5. How can instances be returned as values from functions? Illustrate with a program.
6. Explain the mutability of objects with a relevant example.
7. How is object copying handled in Python? What are shallow and deep copies?
8. Write a program using classes and functions to manipulate time values.
9. Distinguish between pure functions and modifiers in class design.
10. What is the difference between prototyping and planning in object-oriented programming?
11. Explain object-oriented features supported by Python classes and methods.
12. Write a class with a `__str__` method and demonstrate its output.
13. What is the role of the `__init__` method in Python classes?
14. Illustrate operator overloading with an example using the `+` operator.
15. How does type-based dispatch work in object-oriented programming?
16. Define polymorphism with an example.
17. Differentiate between interface and implementation in object-oriented design.
18. Write a program that uses inheritance and polymorphism together.
19. Discuss the advantages of using classes and methods to organize code.
20. What is the importance of printing objects? Show how to customize print output.